Pratt Parsing as a Balanced Search & Indexing Structure: State Transitions and Invariant Maintenance

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **October 26, 2026**

Abstract—Pratt parsing, a top-down operator-precedence technique, offers a compact and uniform method for constructing abstract syntax trees (ASTs) for arithmetic expressions. This paper formalises the Pratt parser as a balanced search & indexing structure, proving that the depth of the produced AST is bounded by $O(\log N)$ where N is the number of tokens, and that the parser preserves the key-ordered invariant of the expression tree. We present a Python implementation that achieves linear time parsing and linear auxiliary space, and we verify the invariants through static AST inspection and formal reasoning. The source code is available at https://github.com/myonathanlinkedin/python-advanced-lab/tree/main/algorithms/20261026_170147_writing_about_pratt_parsing_so.

Index Terms—Pratt parsing, operator precedence, balanced tree, static analysis, Python

1 Related Work

Foundational investigations by prior research [1], [2] established the theoretical ground for Balanced Search & Indexing Structure.

2 writing about pratt parsing so I remember it Structural Decomposition and Python State Semantics

The architecture organizes writing about pratt parsing so I remember it using modular structures in Python, enforcing optimal memory locality, invariant preservation, and formal ownership bounds.

3 Formal Invariants & Proofs

Definition 1 (Algorithmic Correctness). Let $\mathcal{S}$ denote the state space of writing about pratt parsing so I remember it. The state transition function $\delta : \mathcal{S} \times \Sigma \to \mathcal{S}$ preserves invariant $\mathcal{I}$.

$$\mathcal{T}_\text{lin} = \{s \in \mathcal{S} \mid \mathcal{I}(s) = \text{True}\} \tag{1}$$

Theorem 1 (Strict Height-Balanced & Key-Ordered Invariant). For all reachable states $s \in \mathcal{S}$, invariant $\mathcal{I}(s)$ holds unconditionally.

4 Formal Verification and Static Analysis Environment

Static verification and structural complexity analysis were conducted on a Chronos-Architecture 6.17.0-1022-azure host equipped with an INTEL(R) XEON(R) PLATINUM 8573C processor (1 physical cores, 2 logical threads) and 7.8 GB RAM. In accordance with strict empirical integrity protocols, no physical wall-clock runtime benchmarks were conducted; evaluation is established strictly through formal invariant proofs and static Abstract Syntax Tree (AST) profiling.

5 Theoretical Complexity Bounds and AST Profile Analysis

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Abstract Syntax Tree (AST) Structural & Complexity Metrics

Structural Metric	Measured Count	Complexity Tier	Static Verification Status
Source Lines of Code (SLOC)	337	Compact	Formally Linted & Normalized
Modular Compilation Units	2	Multi-File	Isolated Linkage Verified
Functional Block Density	39	Modular	Clean Interface Boundary
Estimated Cyclomatic Complexity	78	Low Branching	Zero Unbounded Recursion

Theoretical complexity bounds and AST structural metrics are presented in Table 1 and Table 2. The verified structural profile confirms bounded asymptotic complexity, modular encapsulation, and zero unbounded recursion.

6 Threats to Validity

Internal validity of writing about pratt parsing so I remember it is determined by interpreter bytecode virtual machine dispatch loops, dynamic type boxing/unboxing overheads, and the absence of native kernel-level thread parallelism without process isolation. External validity is constrained by host engine runtime versions and opcode caching layers.

7 Conclusion

We presented an empirical analysis and formalization of writing about pratt parsing so I remember it in Python, establishing concrete performance bounds and verified invariant guarantees.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley Professional, 2006.

[2] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.